

**Федеральное государственное автономное образовательное
учреждение высшего образования «Национальный
исследовательский университет ИТМО»**

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №3
По Основам Программной Инженерии
Вариант 534117

Выполнил:

Ларионов Владислав Васильевич

Группа Р3209

Проверил:

Ермаков Михаил Константинович

Санкт-Петербург, 2026

Содержание

Задание	3
Дополнительное задание	4
Программа	4
Вывод:	12

Задание

Написать сценарий для утилиты [Apache Ant](#), реализующий компиляцию, тестирование и упаковку в jar-архив кода проекта из [лабораторной работы №3](#) по дисциплине "Веб-программирование".

Каждый этап должен быть выделен в отдельный блок сценария; все переменные и константы, используемые в сценарии, должны быть вынесены в отдельный файл параметров; MANIFEST.MF должен содержать информацию о версии и о запуске классе.

Сценарий должен реализовывать следующие цели (targets):

1. **compile** -- компиляция исходных кодов проекта.
2. **build** -- компиляция исходных кодов проекта и их упаковка в исполняемый jar-архив. Компиляцию исходных кодов реализовать посредством вызова цели **compile**.
3. **clean** -- удаление скомпилированных классов проекта и всех временных файлов (если они есть).
4. **test** -- запуск junit-тестов проекта. Перед запуском тестов необходимо осуществить сборку проекта (цель **build**).
5. **native2ascii** - преобразование [native2ascii](#) для копий файлов локализации (для тестирования сценария все строковые параметры необходимо вынести из классов в файлы локализации).
6. **scp** - перемещение собранного проекта по scp на выбранный сервер по завершению сборки. Предварительно необходимо выполнить сборку проекта (цель **build**).
7. **doc** - добавление в MANIFEST.MF MD5 и SHA-1 файлов проекта, а также генерация и добавление в архив javadoc по всем классам проекта.
8. **xml** - валидация всех xml-файлов в проекте.
9. **music** - воспроизведение музыки по завершению сборки (цель **build**).
10. **team** - осуществляет получение из svn-репозитория 4 предыдущих ревизий, их сборку (по аналогии с основной) и упаковку получившихся jar-файлов в zip-архив. Сборку реализовать посредством вызова цели **build**.
11. **diff** - осуществляет проверку состояния рабочей копии, и, если изменения не касаются классов, указанных в файле параметров выполняет commit в репозиторий git.
12. **env** - осуществляет сборку и запуск программы в альтернативных окружениях; окружение задается версией java и набором аргументов виртуальной машины в файле параметров.
13. **report** - в случае успешного прохождения тестов сохраняет отчет junit в формате xml, добавляет его в репозиторий git и выполняет commit.
14. **alt** - создаёт альтернативную версию программы с измененными именами переменных и классов (используя задание replace/replaceregexp в файлах параметров) и упаковывает её в jar-архив. Для создания jar-архива использует цель **build**.

Дополнительное задание

Реализовать команду ant validate, которая осуществляет проверку хотя бы 90% покрытия программы тестами. При отсутствии выполненного условия сборку не завершать.

Программа

```
<?xml version="1.0" encoding="UTF-8"?>
<project name="lab3-ant" default="build" basedir="."
xmlns:jacoco="antlib:org.jacoco.ant">
  <property file="build.properties"/>

  <path id="compile.classpath">
    <fileset dir="${lib.compile.dir}" includes="*.jar"/>
    <fileset dir="${lib.runtime.dir}" includes="*.jar"/>
  </path>

  <path id="test.classpath">
    <path location="${classes.main.dir}"/>
    <path refid="compile.classpath"/>
    <fileset dir="${lib.test.dir}" includes="*.jar"/>
  </path>

  <path id="test.runtime.classpath">
    <path location="${classes.main.dir}"/>
    <path location="${classes.test.dir}"/>
    <path refid="compile.classpath"/>
  </path>

  <path id="jacoco.classpath">
    <pathelement location="${jacoco.ant.file}"/>
  </path>

  <!-- creates build directories -->
  <target name="init">
    <mkdir dir="${build.dir}"/>
    <mkdir dir="${classes.main.dir}"/>
    <mkdir dir="${classes.test.dir}"/>
    <mkdir dir="${dist.dir}"/>
    <mkdir dir="${lib.compile.dir}"/>
    <mkdir dir="${lib.runtime.dir}"/>
    <mkdir dir="${lib.test.dir}"/>
  </target>

  <!-- downloads required libraries -->
  <target name="resolve" depends="init">
    <get src="${maven.base.url}/jakarta/platform/jakarta.jakartaee-web-
api/${jakartaee.version}/jakarta.jakartaee-web-api-${jakartaee.version}.jar"
        dest="${lib.compile.dir}/jakarta.jakartaee-web-api-
${jakartaee.version}.jar"
```

```

        skipexisting="true"/>

        <get
src="${maven.base.url}/org/primefaces/primefaces/${primefaces.version}/primefaces-
${primefaces.version}-jakarta.jar"
        dest="${lib.runtime.dir}/primefaces-${primefaces.version}-jakarta.jar"
        skipexisting="true"/>

        <get
src="${maven.base.url}/org/postgresql/postgresql/${postgresql.version}/postgresql-
${postgresql.version}.jar"
        dest="${lib.runtime.dir}/postgresql-${postgresql.version}.jar"
        skipexisting="true"/>

        <get
src="${maven.base.url}/org/glassfish/jakarta.faces/${faces.version}/jakarta.faces-
${faces.version}.jar"
        dest="${lib.runtime.dir}/jakarta.faces-${faces.version}.jar"
        skipexisting="true"/>

        <get src="${maven.base.url}/org/glassfish/jakarta.el/${el.version}/jakarta.el-
${el.version}.jar"
        dest="${lib.runtime.dir}/jakarta.el-${el.version}.jar"
        skipexisting="true"/>

        <get src="${maven.base.url}/jakarta/persistence/jakarta.persistence-
api/${persistence.version}/jakarta.persistence-api-${persistence.version}.jar"
        dest="${lib.runtime.dir}/jakarta.persistence-api-
${persistence.version}.jar"
        skipexisting="true"/>

        <get src="${maven.base.url}/jakarta/validation/jakarta.validation-
api/${validation.version}/jakarta.validation-api-${validation.version}.jar"
        dest="${lib.runtime.dir}/jakarta.validation-api-
${validation.version}.jar"
        skipexisting="true"/>

        <get src="${maven.base.url}/org/hibernate/validator/hibernate-
validator/${hibernate.validator.version}/hibernate-validator-
${hibernate.validator.version}.jar"
        dest="${lib.runtime.dir}/hibernate-validator-
${hibernate.validator.version}.jar"
        skipexisting="true"/>

        <get src="${maven.base.url}/org/jboss/logging/jboss-
logging/${jboss.logging.version}/jboss-logging-${jboss.logging.version}.jar"
        dest="${lib.runtime.dir}/jboss-logging-${jboss.logging.version}.jar"
        skipexisting="true"/>

        <get
src="${maven.base.url}/com/fasterxml/classmate/${classmate.version}/classmate-
${classmate.version}.jar"
        dest="${lib.runtime.dir}/classmate-${classmate.version}.jar"
        skipexisting="true"/>

        <get src="${maven.base.url}/jakarta/enterprise/jakarta.enterprise.cdi-
api/${cdi.version}/jakarta.enterprise.cdi-api-${cdi.version}.jar"
        dest="${lib.runtime.dir}/jakarta.enterprise.cdi-api-${cdi.version}.jar"
        skipexisting="true"/>

        <get src="${maven.base.url}/jakarta/interceptor/jakarta.interceptor-
api/${interceptor.version}/jakarta.interceptor-api-${interceptor.version}.jar"

```

```

        dest="${lib.runtime.dir}/jakarta.interceptor-api-
${interceptor.version}.jar"
        skipexisting="true"/>
        <get src="${maven.base.url}/jakarta/servlet/jsp/jstl/jakarta.servlet.jsp.jstl-
api/${jstl.api.version}/jakarta.servlet.jsp.jstl-api-${jstl.api.version}.jar"
        dest="${lib.runtime.dir}/jakarta.servlet.jsp.jstl-api-
${jstl.api.version}.jar"
        skipexisting="true"/>
        <get
src="${maven.base.url}/org/glassfish/web/jakarta.servlet.jsp.jstl/${jstl.impl.version}
/jakarta.servlet.jsp.jstl-${jstl.impl.version}.jar"
        dest="${lib.runtime.dir}/jakarta.servlet.jsp.jstl-
${jstl.impl.version}.jar"
        skipexisting="true"/>

        <get src="${maven.base.url}/org/junit/platform/junit-platform-console-
standalone/${junit.console.version}/junit-platform-console-standalone-
${junit.console.version}.jar"
        dest="${lib.test.dir}/junit-platform-console-standalone-
${junit.console.version}.jar"
        skipexisting="true"/>

        <get
src="${maven.base.url}/org/jacoco/org.jacoco.ant/${jacoco.version}/org.jacoco.ant-
${jacoco.version}-nodeps.jar"
        dest="${jacoco.ant.file}"
        skipexisting="true"/>
    </target>

    <!-- loads JaCoCo Ant tasks -->
    <target name="jacoco-taskdef" depends="resolve">
        <taskdef uri="antlib:org.jacoco.ant"
            resource="org/jacoco/ant/antlib.xml">
            <classpath refid="jacoco.classpath"/>
        </taskdef>
    </target>

    <!-- creates MANIFEST.MF -->
    <target name="manifest" depends="init">
        <manifest file="${manifest.file}">
            <attribute name="Manifest-Version" value="1.0"/>
            <attribute name="Implementation-Title" value="${project.name}"/>
            <attribute name="Implementation-Version" value="${project.version}"/>
            <attribute name="Main-Class" value="${main.class}"/>
            <attribute name="Built-By" value="${user.name}"/>
        </manifest>
    </target>

    <!-- compile -->
    <target name="compile" depends="resolve">
        <javac srcdir="${src.main.dir}"
            destdir="${classes.main.dir}"
            release="${java.release}"

```

```

        encoding="${encoding}"
        includeantruntime="false">
        <classpath refid="compile.classpath"/>
        <compilerarg value="-parameters"/>
    </javac>
    <copy todir="${classes.main.dir}">
        <fileset dir="${resources.dir}"/>
    </copy>
</target>

<!-- build -->
<target name="build" depends="compile,manifest">
    <jar destfile="${jar.file}" basedir="${classes.main.dir}"
manifest="${manifest.file}"/>
</target>

<!-- build_war -->
<target name="build_war" depends="compile,manifest">
    <war destfile="${war.file}" webxml="${webapp.dir}/WEB-INF/web.xml"
manifest="${manifest.file}">
        <fileset dir="${webapp.dir}" excludes="WEB-INF/web.xml"/>
        <classes dir="${classes.main.dir}"/>
        <lib dir="${lib.runtime.dir}" includes="*.jar"/>
    </war>
</target>

<!-- clean -->
<target name="clean">
    <delete dir="${build.dir}"/>
    <delete dir="${dist.dir}"/>
</target>

<!-- compile JUnit tests -->
<target name="compile-tests" depends="compile">
    <delete dir="${classes.test.dir}"/>
    <mkdir dir="${classes.test.dir}"/>
    <javac srcdir="${src.test.dir}"
        destdir="${classes.test.dir}"
        release="${java.release}"
        encoding="${encoding}"
        includeantruntime="false">
        <classpath refid="test.classpath"/>
    </javac>
</target>

<!-- test -->
<target name="test" depends="build,compile-tests">
    <mkdir dir="${reports.dir}/junit"/>
    <pathconvert property="test.runtime.classpath.value"
refid="test.runtime.classpath"/>
    <java jar="${lib.test.dir}/junit-platform-console-standalone-
${junit.console.version}.jar"
        fork="true"

```

```

        failonerror="true">
        <arg value="--class-path"/>
        <arg value="\${test.runtime.classpath.value}"/>
        <arg value="--scan-class-path"/>
        <arg value="--reports-dir"/>
        <arg value="\${reports.dir}/junit"/>
    </java>
</target>

<!-- native2ascii -->
<target name="native2ascii" depends="init">
    <mkdir dir="\${native2ascii.dir}"/>
    <native2ascii src="\${resources.dir}"
        dest="\${native2ascii.dir}"
        encoding="\${encoding}"
        includes="messages_ru.properties"/>
</target>

<!-- scp -->
<target name="scp" depends="build,build_war">
    <exec executable="scp" failonerror="true">
        <arg value="\${war.file}"/>
        <arg value="\${scp.user}@{\scp.host}:{\scp.remote.dir}"/>
    </exec>
</target>

<!-- doc -->
<target name="doc" depends="build">
    <mkdir dir="\${javadoc.dir}"/>
    <javadoc sourcepath="\${src.main.dir}"
        destdir="\${javadoc.dir}"
        classpathref="compile.classpath"
        encoding="\${encoding}"
        docencoding="\${encoding}"
        charset="\${encoding}"
        source="\${java.release}"
        additionalparam="-Xdoclint:none"/>
    <mkdir dir="\${build.dir}/checksums"/>
    <checksum algorithm="MD5" totalproperty="project.md5"
todir="\${build.dir}/checksums">
        <fileset dir="\${src.main.dir}" includes="**/*.java"/>
    </checksum>
    <checksum algorithm="SHA-1" totalproperty="project.sha1"
todir="\${build.dir}/checksums">
        <fileset dir="\${src.main.dir}" includes="**/*.java"/>
    </checksum>
    <manifest file="\${manifest.doc.file}">
        <attribute name="Manifest-Version" value="1.0"/>
        <attribute name="Implementation-Title" value="\${project.name}"/>
        <attribute name="Implementation-Version" value="\${project.version}"/>
        <attribute name="Main-Class" value="\${main.class}"/>
        <attribute name="Project-MD5" value="\${project.md5}"/>
        <attribute name="Project-SHA1" value="\${project.sha1}"/>
    </manifest>
</target>

```



```

        </manifest>
        <jar destfile="${jar.file}" update="true" manifest="${manifest.doc.file}">
            <zipfileset dir="${javadoc.dir}" prefix="javadoc"/>
        </jar>
    </target>

    <!-- xml -->
    <target name="xml">
        <xmlvalidate failonerror="true" lenient="true">
            <fileset dir="${webapp.dir}" includes="WEB-INF/*.xml"/>
        </xmlvalidate>
    </target>

    <!-- music -->
    <target name="music" depends="build">
        <exec executable="${music.command}" failonerror="false">
            <arg value="${music.file}"/>
        </exec>
    </target>

    <!-- team -->
    <target name="team">
        <mkdir dir="${team.dir}"/>
        <echo file="${team.dir}/build-team.sh"><![CDATA[
set -e

rm -rf "${team.dir}/checkouts" "${team.dir}/jars"
mkdir -p "${team.dir}/checkouts" "${team.dir}/jars"

svn log -q -l "${team.revision.count}" "${svn.url}" \
| awk '/^r[0-9]+/ {print substr($1, 2)}' \
| while read rev; do
    checkout="${team.dir}/checkouts/r$rev"

    svn checkout -q -r "$rev" "${svn.url}" "$checkout"
    rm -rf "$checkout/${lib.dir}"
    cp -R "${lib.dir}" "$checkout/${lib.dir}"

    (cd "$checkout" && ant build)
    cp "$checkout/${jar.file}" "${team.dir}/jars/${project.name}-r$rev.jar"
done
]]></echo>
        <exec executable="sh" failonerror="true">
            <arg value="${team.dir}/build-team.sh"/>
        </exec>
        <zip destfile="${team.zip}" basedir="${team.dir}/jars"/>
    </target>

    <!-- diff -->
    <target name="diff">
        <echo file="${build.dir}/diff.sh"><![CDATA[
set -e

```

```

changed=$(git status --porcelain | awk '{print $2}')

if [ -z "$changed" ]; then
    echo "No changes to commit"
    exit 0
fi

if echo "$changed" | grep -E "${diff.protected.pattern}"; then
    echo "Protected class changes detected, commit skipped"
    exit 1
fi

git add -A
git commit -m "${diff.commit.message}"
]]></echo>
    <exec executable="sh" failonerror="true">
        <arg value="${build.dir}/diff.sh"/>
    </exec>
</target>

<!-- env -->
<target name="env" depends="build">
    <exec executable="${env.java.executable}" failonerror="true">
        <arg line="${env.jvm.args}"/>
        <arg value="-jar"/>
        <arg value="${jar.file}"/>
        <arg line="${env.program.args}"/>
    </exec>
</target>

<!-- report -->
<target name="report" depends="test">
    <exec executable="sh" failonerror="false">
        <arg value="-c"/>
        <arg value="git add -f '${reports.dir}/junit' && git commit -m
'${report.commit.message}'"/>
    </exec>
</target>

<!-- alt -->
<target name="alt">
    <delete dir="${alt.work.dir}"/>
    <copy todir="${alt.work.dir}">
        <fileset dir=".">
            <exclude name="build/**"/>
            <exclude name="${build.dir}/**"/>
            <exclude name="${dist.dir}/**"/>
            <exclude name=".git/**"/>
            <exclude name=".svn/**"/>
        </fileset>
    </copy>
    <replaceregexp match="${alt.class.old}" replace="${alt.class.new}" flags="g"
byline="true">

```

```

        <fileset dir="${alt.work.dir}/src/main/java" includes="**/*.java"/>
    </replaceregexp>
    <replaceregexp match="${alt.var.old}" replace="${alt.var.new}" flags="g"
byline="true">
        <fileset dir="${alt.work.dir}/src/main/java" includes="**/*.java"/>
    </replaceregexp>
    <move
file="${alt.work.dir}/src/main/java/org/example/entities/${alt.class.old}.java"
tofile="${alt.work.dir}/src/main/java/org/example/entities/${alt.class.new}.java"/>
    <ant dir="${alt.work.dir}" target="build" inheritAll="false">
        <property name="project.name" value="${project.name}-alt"/>
    </ant>
    <copy file="${alt.work.dir}/${alt.jar}" tofile="${alt.jar}"/>
</target>

<!-- validate -->
<target name="validate" depends="build,compile-tests,jacoco-taskdef">
    <delete file="${jacoco.exec.file}"/>
    <delete dir="${jacoco.report.dir}"/>
    <mkdir dir="${reports.dir}/junit"/>
    <mkdir dir="${jacoco.report.dir}"/>
    <pathconvert property="test.runtime.classpath.value"
refid="test.runtime.classpath"/>
    <jacoco:coverage destfile="${jacoco.exec.file}" append="false">
        <java jar="${lib.test.dir}/junit-platform-console-standalone-
${junit.console.version}.jar"
            fork="true"
            failonerror="true">
            <arg value="--class-path"/>
            <arg value="${test.runtime.classpath.value}"/>
            <arg value="--scan-class-path"/>
            <arg value="--reports-dir"/>
            <arg value="${reports.dir}/junit"/>
        </java>
    </jacoco:coverage>
    <jacoco:report>
        <executiondata>
            <file file="${jacoco.exec.file}"/>
        </executiondata>
        <structure name="${project.name}">
            <classfiles>
                <fileset dir="${classes.main.dir}" includes="**/*.class"/>
            </classfiles>
            <sourcefiles encoding="${encoding}">
                <fileset dir="${src.main.dir}" includes="**/*.java"/>
            </sourcefiles>
        </structure>
        <html destdir="${jacoco.report.dir}/html"/>
        <xml destfile="${jacoco.report.xml}"/>
        <check failonviolation="true">
            <rule element="BUNDLE">

```

```
        <limit counter="INSTRUCTION" value="COVEREDRATIO"  
minimum="${jacoco.coverage.minimum}"/>  
        </rule>  
    </check>  
    </jacoco:report>  
</target>  
  
</project>
```

Вывод:

Во время выполнения данной лабораторной работы я:

- 1) Научился работать с системой сборки Ant
- 2) Повторил навыки работы с Make, Gradle, Maven
- 3) Изучил основы тестирования ПО
- 4) Разобрался со всеми видами тестирования и зачем они нужны
- 5) Покрыл программу 90% тестами